LAPORAN PRAKTIKUM

“Queue”

STRUKTUR DATA

DISUSUN OLEH:

NOFRI ILHAM

2511531013

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data dengan materi Queue ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum serta sebagai sarana untuk memahami konsep struktur data Queue yang menerapkan prinsip First In First Out (FIFO). Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pengalaman, khususnya dalam memahami konsep, operasi dasar, serta implementasi Queue dalam pemrograman.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi isi maupun penyajiannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat bagi penulis sendiri maupun bagi pembaca dalam memahami materi Queue pada struktur data

Wassalamu'alaikum Warahmatullahi Wabarakatuh

Padang, 29 April 2026

Nofri Ilham

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II PEMBAHASAN 3
2.1 Pengertian Queue 3
2.2 Fungsi Queue 3
2.3 Impementasi Queue pada Praktikum 5
2.3.1 Kode Pertama(QueueArray_2511531013) 5
2.3.2 Kode Kedua(QueueArrayDriver_2511531013) 7
2.3.3 Kode Ketiga(IterasiQueue_2511531013) 8
2.3.4 Kode Keempat(QueueLinkedList_2511531013) 9
2.3.5 Kode Kelima(reverseData_2511531013) 10
BAB III PENUTUP 12
3.1 Kesimpulan 12
3.2 Saran 12
DAFTAR PUSTAKA 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam perkembangan ilmu komputer, struktur data memegang peranan penting dalam pengelolaan dan pengolahan data secara efisien. Salah satu struktur data yang sering digunakan adalah Queue. Queue merupakan struktur data yang menerapkan prinsip First In First Out (FIFO), yaitu elemen yang pertama kali masuk akan menjadi elemen pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari, seperti antrian di loket pelayanan, sistem penjadwalan proses pada sistem operasi, serta pengelolaan data pada jaringan komputer.

Pemahaman mengenai Queue sangat penting bagi mahasiswa, khususnya dalam bidang pemrograman dan pengembangan sistem, karena struktur ini membantu dalam menyelesaikan berbagai permasalahan yang berkaitan dengan pengaturan urutan data. Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar Queue, cara implementasinya dalam bahasa pemrograman, serta mampu mengaplikasikannya dalam penyelesaian masalah secara nyata.

Selain itu, praktikum ini juga bertujuan untuk melatih kemampuan analisis dan logika mahasiswa dalam mengelola data secara terstruktur dan sistematis. Dengan demikian, pemahaman terhadap Queue tidak hanya bersifat teoritis, tetapi juga dapat diterapkan secara praktis dalam pengembangan perangkat lunak.

1.2 Tujuan

Adapun tujuan dari praktikum struktur data dengan materi *Queue* adalah sebagai berikut:

1. Memahami konsep dasar struktur data Queue beserta prinsip kerjanya yaitu First In First Out (FIFO).
2. Mengetahui operasi-operasi dasar pada Queue, seperti enqueue, dequeue, peek, dan pengecekan kondisi empty maupun full.
3. Mampu mengimplementasikan struktur data Queue menggunakan bahasa pemrograman.
4. Melatih kemampuan logika dan analisis dalam menyelesaikan permasalahan menggunakan konsep Queue.
5. Menerapkan penggunaan Queue dalam kasus nyata, seperti simulasi antrian atau pengolahan data secara berurutan.

1.3 Manfaat

Adapun manfaat dari praktikum struktur data dengan materi *Queue* adalah sebagai berikut:

1. Membantu mahasiswa memahami penerapan konsep Queue dalam pengelolaan data secara berurutan.
2. Meningkatkan kemampuan pemrograman melalui implementasi struktur data Queue dalam bahasa pemrograman.
3. Melatih pola pikir logis dan sistematis dalam menyelesaikan masalah yang berkaitan dengan antrian data.
4. Memberikan gambaran penerapan Queue dalam kehidupan nyata, seperti sistem antrian layanan dan penjadwalan proses.
5. Menjadi dasar pemahaman untuk mempelajari struktur data lain yang lebih kompleks.

BAB II
PEMBAHASAN

2.1 Pengertian Queue

Queue adalah struktur data yang menggunakan prinsip FIFO (First In First Out), di mana elemen pertama yang dimasukkan ke dalam queue akan menjadi elemen pertama yang dikeluarkan.

Prinsip ini mirip dengan antrean pada umumnya, seperti antrean di kasir atau saat mencetak dokumen. Dalam pemrograman, queue efektif untuk mengatur data atau task yang diproses secara berurutan, memastikan semua operasi dijalankan secara efisien.

Queue juga dapat digunakan dalam berbagai bentuk, termasuk simple queue, circular queue, priority queue, dan double-ended queue (deque), masing-masing dengan karakteristik dan kegunaan khusus sesuai dengan kebutuhan aplikasi.

2.2 Fungsi Queue

Berikut beberapa fungsi utama dari queue:

1. Organisasi Data

   Queue membantu menyusun data atau tugas dalam urutan yang spesifik, biasanya berdasarkan waktu kedatangan. Hal ini memudahkan penanganan tugas sesuai urutan yang telah ditetapkan, menghindari kekacauan dan memastikan proses berjalan lancar.

2. Pengelolaan sumber daya

   Dalam sistem yang menggunakan sumber daya bersama (seperti printer atau CPU), queue memastikan setiap permintaan diproses secara adil dan berurutan. Ini menghindari konflik dan memaksimalkan efisiensi penggunaan sumber daya.

3. Peforma sistem

Dengan menggunakan queue, sistem bisa mengelola tugas-tugas dengan lebih efisien. Misalnya, dalam sistem operasi, queue dimanfaatkan untuk penjadwalan tugas sehingga mengurangi waktu tunggu dan meningkatkan responsivitas sistem.

4. Peningkatan troughput

   Queue memungkinkan sistem menangani work load tinggi dengan lebih efektif, sebab dapat menyimpan tugas sementara hingga sumber daya yang diperlukan tersedia. Hal ini khususnya penting dalam web app dan database yang melayani banyak permintaan secara bersamaan.

5. Kemudahan pemrograman

   Queue menyederhanakan pengembangan aplikasi yang membutuhkan pengolahan tugas atau data secara berurutan. Dengan mengikuti prinsip FIFO, developer lebih mudah mengimplementasikan dan debugging aplikasi.

6. Asinkron dan komunikasi

   Dalam pengembangan aplikasi modern, queue berfungsi untuk mengelola pesan atau tugas yang harus diproses asinkron. Misalnya, dalam aplikasi yang menggunakan microservices, queue dipakai untuk komunikasi antar-service yang efisien dan terorganisasi.

2.3 Impementasi Queue pada Praktikum

2.3.1 Kode Pertama(QueueArray_2511531013)

```
54                  return;
55              }
56              //Kunjungi dari belakang dan cetak
57              for(i=front_1013;i<rear_1013;i++) {
58                  System.out.printf("%d <--",array_1013[i]);
59              }
60              return;
61          }
62
63
64
65
66
67
68
69
70
71  }
72
```

Kode Program 2.1

Kode Program QueueArray_2511531013 merupakan implementasi struktur data Queue menggunakan array dengan konsep circular queue. Queue adalah struktur data yang menggunakan prinsip First In First Out (FIFO), yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Pada program ini terdapat beberapa atribut utama, yaitu front_1013 yang berfungsi sebagai penunjuk elemen paling depan, rear_1013 sebagai penunjuk elemen paling belakang, size_1013 untuk menyimpan jumlah elemen dalam antrian, capacity_1013 sebagai kapasitas maksimum antrian, serta array_1013[] sebagai media penyimpanan data.

Constructor QueueArray_2511531013(int capacity) digunakan untuk menginisialisasi kapasitas antrian, menetapkan nilai awal front dan size sebesar 0, serta rear diatur ke capacity – 1 agar proses penambahan data pertama berjalan dengan benar. Selain itu, array juga dibuat sesuai kapasitas yang ditentukan.

Method isFull_2511531013() berfungsi untuk mengecek apakah antrian sudah penuh, yaitu ketika jumlah elemen sama dengan kapasitas. Sedangkan method isEmpty_2511531013() digunakan untuk mengecek apakah antrian kosong, yaitu ketika jumlah elemen sama dengan nol.

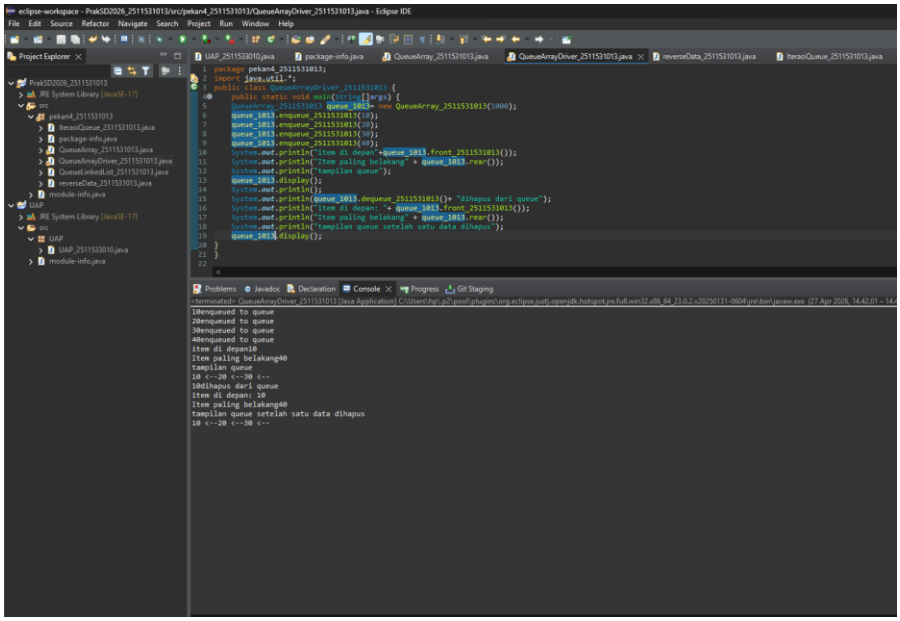
Method enqueue_2511531013(int item) digunakan untuk menambahkan elemen ke dalam antrian. Jika antrian belum penuh, maka nilai rear akan diperbarui menggunakan operasi modulo agar bersifat melingkar, kemudian elemen dimasukkan ke dalam array dan size ditambah satu.

Method dequeue_2511531013() berfungsi untuk menghapus elemen dari bagian depan antrian. Jika antrian kosong, maka akan mengembalikan nilai minimum integer. Jika tidak kosong, elemen pada posisi front diambil, lalu indeks front seharusnya diperbarui secara melingkar dan size dikurangi satu.

Method front_2511531013() digunakan untuk melihat elemen paling depan tanpa menghapusnya, sedangkan method rear() digunakan untuk melihat elemen paling belakang.

Method display() berfungsi untuk menampilkan isi antrian dari depan ke belakang. Jika antrian kosong, maka akan menampilkan pesan bahwa antrian kosong.

2.3.2 Kode Kedua(QueueArrayDriver_2511531013)



Kode Program 2.2

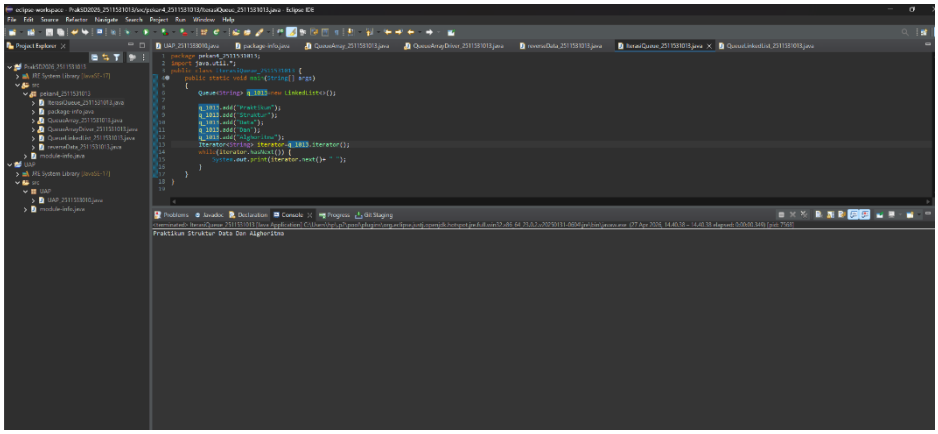
Program QueueArrayDriver_2511531013 merupakan kelas utama (driver class) yang digunakan untuk menguji implementasi struktur data Queue yang telah dibuat pada kelas QueueArray_2511531013. Program ini berfungsi untuk menjalankan operasi-operasi dasar pada Queue seperti penambahan (enqueue), penghapusan (dequeue), serta menampilkan elemen antrian.

Pada bagian main, pertama dibuat objek queue_1013 dengan kapasitas sebesar 1000 elemen. Selanjutnya dilakukan beberapa operasi enqueue untuk menambahkan data ke dalam antrian, yaitu nilai 10, 20, 30, dan 40. Data tersebut akan tersimpan secara berurutan sesuai prinsip First In First Out (FIFO).

Setelah itu, program menampilkan elemen paling depan menggunakan method front_2511531013() dan elemen paling belakang menggunakan method rear(). Kemudian, isi antrian ditampilkan menggunakan method display().

Selanjutnya dilakukan operasi dequeue_2511531013() untuk menghapus satu elemen dari bagian depan antrian. Karena menggunakan prinsip FIFO, maka elemen pertama yang dimasukkan (yaitu 10) akan menjadi elemen yang dihapus. Setelah penghapusan, program kembali menampilkan elemen terdepan, elemen terakhir, serta isi antrian untuk melihat perubahan yang terjadi.

2.3.3 Kode Ketiga(IterasiQueue_2511531013)



Kode Program 2.3

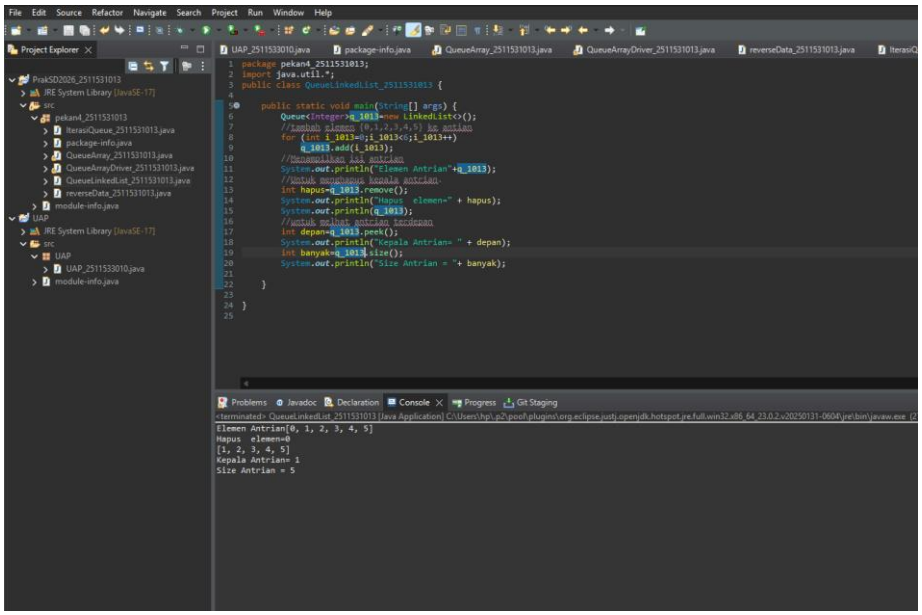
Program IterasiQueue_2511531013 adalah contoh penggunaan struktur data Queue yang diimplementasikan menggunakan kelas LinkedList dari Java Collection Framework. Pada program ini, Queue digunakan untuk menyimpan data bertipe String.

Pada method main, pertama dibuat objek Queue bernama q_1013 dengan menggunakan LinkedList. Setelah itu, beberapa data dimasukkan ke dalam Queue menggunakan method add(), yaitu “Praktikum”, “Struktur”, “Data”, “Dan”, dan “Alghoritma”. Data yang dimasukkan akan tersusun sesuai prinsip First In First Out (FIFO).

Selanjutnya, program menggunakan Iterator untuk melakukan iterasi atau penelusuran terhadap seluruh elemen dalam

Queue. Objek iterator dibuat dengan method iterator(), kemudian dilakukan perulangan menggunakan while (iterator.hasNext()). Selama masih ada elemen berikutnya, method next() akan mengambil dan menampilkan elemen tersebut satu per satu.

2.3.4 Kode Keempat(QueueLinkedList_2511531013)



Kode program 2.4

Program QueueLinkedList_2511531013 merupakan implementasi struktur data Queue dengan memanfaatkan kelas LinkedList dari Java Collection Framework. Pada program ini, Queue digunakan untuk menyimpan data bertipe Integer.

Pada method main, pertama dibuat objek Queue bernama q_1013. Selanjutnya dilakukan proses penambahan elemen ke dalam antrian menggunakan perulangan for, yaitu menambahkan angka dari 0 sampai 5. Data yang dimasukkan akan tersusun sesuai prinsip First In First Out (FIFO).

Setelah itu, program menampilkan seluruh isi antrian menggunakan perintah System.out.println, ko terlihat semua elemen yang telah dimasukkan.

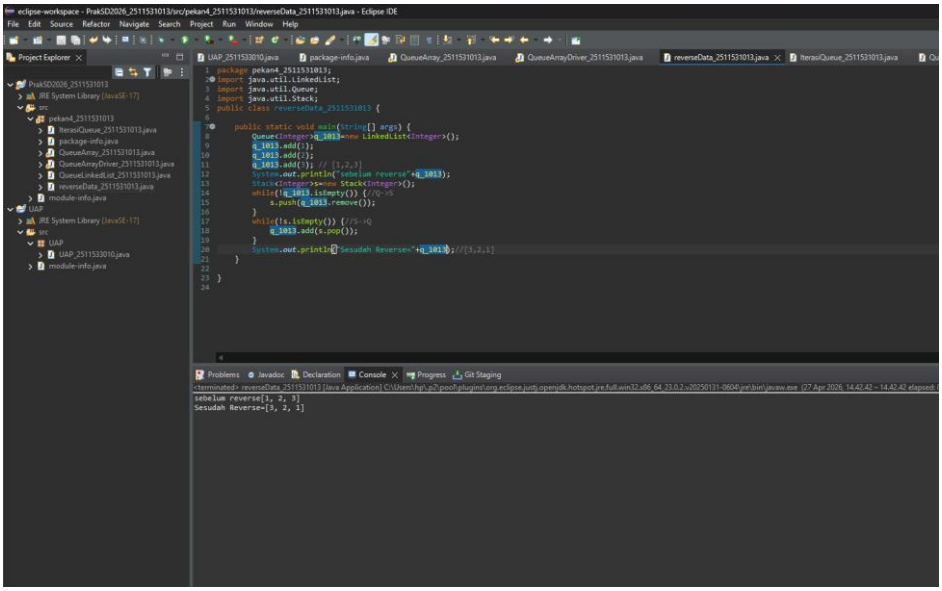
Kemudian dilakukan operasi remove() untuk menghapus elemen paling depan (kepala antrian). Nilai yang dihapus disimpan dalam variabel hapus dan ditampilkan ke layar. Setelah penghapusan, isi antrian kembali ditampilkan untuk melihat perubahan yang terjadi.

Selanjutnya, method peek() digunakan untuk melihat elemen terdepan tanpa menghapusnya. Nilai tersebut disimpan dalam variabel depan dan ditampilkan.

Terakhir, method size() digunakan untuk mengetahui jumlah elemen yang tersisa dalam antrian, kemudian hasilnya ditampilkan ke layar.

Secara keseluruhan, program ini menunjukkan penggunaan Queue dengan LinkedList yang menyediakan berbagai operasi dasar seperti penambahan (add), penghapusan (remove), melihat elemen depan (peek), serta menghitung jumlah elemen (size) secara sederhana dan efisien.

2.3.5 Kode Kelima(reverseData_2511531013)



Kode Program 2.5

Program reverseData_2511531013 bertujuan untuk membalik urutan elemen dalam struktur data Queue dengan bantuan struktur data Stack. Program ini memanfaatkan perbedaan prinsip antara Queue yang menggunakan First In First Out (FIFO) dan Stack yang menggunakan Last In First Out (LIFO).

Pada method main, pertama dibuat objek Queue bernama q_1013 menggunakan LinkedList. Kemudian dimasukkan beberapa elemen, yaitu 1, 2, dan 3, sehingga urutan awal antrian adalah [1, 2, 3]. Setelah itu, isi antrian ditampilkan sebelum proses pembalikan.

Selanjutnya dibuat objek Stack bernama s. Proses pembalikan dilakukan dalam dua tahap. Tahap pertama adalah memindahkan seluruh elemen dari Queue ke Stack menggunakan perulangan while (!q_1013.isEmpty()). Karena Stack bersifat LIFO, urutan elemen akan terbalik saat dimasukkan ke dalamnya.

Tahap kedua adalah memindahkan kembali elemen dari Stack ke Queue menggunakan perulangan while (!s.isEmpty()). Proses ini menghasilkan urutan yang sudah terbalik dalam Queue.

Akhirnya, isi Queue ditampilkan kembali setelah proses reverse, yang menunjukkan perubahan dari [1, 2, 3] menjadi [3, 2, 1].

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Queue merupakan struktur data linear yang bekerja dengan prinsip First In First Out (FIFO), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar . Dalam implementasinya, Queue memiliki beberapa operasi dasar seperti enqueue (menambah data), dequeue (menghapus data), serta operasi untuk melihat elemen depan dan belakang.

Pada praktikum ini, Queue berhasil diimplementasikan menggunakan dua cara, yaitu dengan array (circular queue) dan LinkedList. Selain itu, dilakukan juga beberapa percobaan seperti iterasi Queue menggunakan Iterator serta pembalikan data (reverse) dengan bantuan Stack. Hal ini menunjukkan bahwa struktur data Queue sangat fleksibel dan dapat dikombinasikan dengan struktur data lain untuk menyelesaikan berbagai permasalahan.

Secara keseluruhan, praktikum ini memberikan pemahaman yang baik mengenai cara kerja Queue, implementasi dalam pemrograman, serta penerapannya dalam kehidupan nyata seperti sistem antrian, penjadwalan proses, dan pengolahan data secara berurutan

3.2 Saran

Adapun saran yang dapat diberikan dari praktikum ini adalah sebagai berikut:

1. Mahasiswa diharapkan lebih sering berlatih mengimplementasikan struktur data Queue dalam berbagai kasus agar pemahaman semakin mendalam.
2. Perlu dilakukan pengembangan program dengan kasus yang lebih kompleks, seperti priority queue atau simulasi antrian nyata.

3. Mahasiswa disarankan untuk memahami perbedaan implementasi Queue menggunakan array dan LinkedList agar dapat memilih metode yang paling sesuai dengan kebutuhan.
4. Praktikum selanjutnya diharapkan dapat menggabungkan Queue dengan struktur data lain untuk meningkatkan kemampuan pemecahan masalah dalam pemrograman.

DAFTAR PUSTAKA

[1]RevoU, “Queue,” RevoU Kosakata Queue. [Daring]. Tersedia pada: https://www.revou.co/kosakata/queue [Diakses: 29-Apr-2026].

LAPORAN TUGAS PEKAN 3

STRUKTUR DATA

DISUSUN OLEH:

NOFRI ILHAM

2511531013

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

1. AntrianLoket_2511531013

```java
package pekan4_2511531013;
import java.util.*;

public class AntrianLoket_2511531013 {
    //Deklarasi Variabel
    String queue_1013[]; // array penyimpan data
    int front_1013;      // indeks depan
    int rear_1013;       // indeks belakang
    int max_1013;        // kapasitas maksimum

    // Constructor
    public AntrianLoket_2511531013(int max_1013) {
        this.max_1013 = max_1013;
        queue_1013 = new String[max_1013];
        front_1013 = -1;
        rear_1013 = -1;
    }

    // Cek kosong
    boolean isEmpty_1013() {
        return (front_1013 == -1);
    }

    // Cek penuh
    boolean isFull_1013() {
        return (rear_1013 == max_1013 - 1);
    }

    // Enqueue (menambah antrian)
    void enqueue_1013(String data_1013) {
        if (isFull_1013()) {
            System.out.println("Antrian penuh!");
        } else {
            if (isEmpty_1013()) {
                front_1013 = 0;
            }
            rear_1013++;
            queue_1013[rear_1013] = data_1013;
            System.out.println("Data berhasil ditambahkan ke antrian");
        }
    }

    // Dequeue_1013 (hapus data)
    void dequeue_1013() {
        if (isEmpty_1013()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println(queue_1013[front_1013] + " telah dilayani");
            front_1013++;

            // reset jika kosong lagi
            if (front_1013 > rear_1013) {
                front_1013 = rear_1013 = -1;
            }
        }
    }

    // Display (tampilkan antrian)
    void display_1013() {
        if (isEmpty_1013()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Isi antrian:");
            int no_1013 = 1;
            for (int i_1013 = front_1013; i_1013 <= rear_1013; i_1013++) {
                System.out.println(no_1013 + ". " + queue_1013[i_1013]);
                no_1013++;
            }
        }
    }

    // Reverse antrian
    void reverse_1013() {
        if (isEmpty_1013()) {
            System.out.println("Antrian kosong!");
        } else {
            int i_1013 = front_1013, j_1013 = rear_1013;
            while (i_1013 < j_1013) {
                String temp_1013 = queue_1013[i_1013];
                queue_1013[i_1013] = queue_1013[j_1013];
                queue_1013[j_1013] = temp_1013;
                i_1013++;
                j_1013--;
            }
            System.out.println("Antrian berhasil dibalik");
        }
    }

    // Main Program (Menu)
    public static void main(String[] args) {
        Scanner input_1013 = new Scanner(System.in);
        AntrianLoket_2511531013 q_1013 = new AntrianLoket_2511531013(10);

        int pilihan_1013;
        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan_1013 = input_1013.nextInt();
            input_1013.nextLine(); // buang enter

            switch (pilihan_1013) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama_1013 = input_1013.nextLine();
                    q_1013.enqueue_1013(nama_1013);
                    break;
                case 2:
                    q_1013.dequeue_1013();
                    break;
                case 3:
                    q_1013.display_1013();
                    break;
                case 4:
                    q_1013.reverse_1013();
                    break;
                case 5:
                    System.out.println("Program selesai");
                    break;
                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilihan_1013 != 5);
        input_1013.close();    }
}
```

Pada bagian awal, dideklarasikan variabel queue_1013[] sebagai array penyimpan data, front_1013 sebagai indeks elemen terdepan, rear_1013 sebagai indeks elemen terakhir, serta max_1013 sebagai kapasitas maksimum antrian.

Constructor AntrianLoket_2511531013(int max_1013) digunakan untuk menginisialisasi kapasitas antrian, membuat array sesuai kapasitas, serta mengatur nilai awal front_1013 dan rear_1013 menjadi -1 yang menandakan antrian masih kosong.

Method isEmpty_1013() burnings untuk mengecek apakah antrian kosong dengan kondisi front_1013 == -1, sedangkan isFull_1013() digunakan untuk mengecek apakah antrian penuh dengan kondisi rear_1013 == max_1013 – 1.

Method enqueue_1013(String data_1013) digunakan untuk menambahkan data ke dalam antrian. Jika antrian penuh, maka akan menampilkan pesan. Jika tidak, maka rear_1013 akan ditambah satu dan data dimasukkan ke dalam array. Jika sebelumnya antrian kosong, maka front_1013 diatur menjadi 0.

Method dequeue_1013() digunakan untuk menghapus data dari bagian depan antrian. Jika antrian kosong, maka akan menampilkan pesan. Jika tidak, data pada indeks front_1013 akan diambil dan front_1013 dinaikkan. Jika setelah penghapusan antrian menjadi kosong, maka front_1013 dan rear_1013 akan di-reset ke -1.

Method display_1013() berfungsi untuk menampilkan seluruh isi antrian dari depan ke belakang menggunakan perulangan dari front_1013 hingga rear_1013.

Method reverse_1013() digunakan untuk membalik urutan elemen dalam antrian dengan cara menukar elemen dari depan dan belakang secara bertahap hingga posisi bertemu di tengah.

Pada bagian main, program menyediakan menu interaktif yang memungkinkan pengguna memilih operasi seperti menambah antrian, menghapus antrian, menampilkan isi antrian, membalik antrian, dan keluar dari program. Input pengguna dibaca menggunakan Scanner, kemudian diproses menggunakan struktur switch-case.

2. Output

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Andi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Budi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
2. Budi

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Budi
2. Andi

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Budi telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```